

Python3 for DSA — Quick Reference

Lists (Arrays)

1. `arr = [1, 2, 3, 4, 5]`
2. `arr.append(6)` # add to end
3. `arr.pop()` # remove & return last element
4. `arr.pop(0)` # remove & return element at index 0
5. `arr.insert(2, 99)` # insert 99 at index 2
6. `arr[::-1]` # reverse (creates new list)
7. `arr.reverse()` # reverse in-place
8. `len(arr)` # length
9. `arr[1:4]` # slicing: index 1 to 3 (4 excluded)
10. `sorted(arr)` # returns new sorted list
11. `arr.sort()` # sorts in-place
12. `arr.sort(reverse=True)` # descending
13. `max(arr), min(arr), sum(arr)`
14. `arr.index(3)` # first index of value 3
15. `99 in arr` # membership check → True/False

Strings

1. `s = "hello"`
2. `s[::-1]` # reverse a string
3. `s.lower(), s.upper()`
4. `s.strip()` # remove leading/trailing whitespace
5. `s.split(",")` # split into list
6. `"-".join(["a", "b", "c"])` # join list into string → "a-b-c"
7. `s.replace("l", "L")`
8. `ord('a'), chr(97)` # char <-> ASCII
9. `s.isalpha(), s.isdigit()`

Important: strings are immutable — `s[0] = 'H'` will error. Convert to list first: `list(s)`, then `"".join(list_of_chars)` to convert back.

Dictionaries (Hashmaps) — your most-used tool

1. `d = {}`
2. `d["key"] = "value"`
3. `d.get("key", 0)` # returns 0 if "key" not found (avoids KeyError)
4. `"key" in d` # membership check
5. `d.keys(), d.values(), d.items()`
6. `for k, v in d.items():`
 `print(k, v)`

7. from collections import defaultdict
d = defaultdict(int) # auto-initializes missing keys to 0
d["x"] += 1 # no KeyError even if "x" wasn't there before
8. from collections import Counter
freq = Counter(arr) # instantly get frequency count of all elements

Sets (unique elements, fast lookup)

9. s = set()
10. s.add(5)
11. s.remove(5) # errors if not present
12. s.discard(5) # safe, no error if not present
13. 5 in s # O(1) lookup — great for "have I seen this?"

Common built-in functions

1. range(5) # 0,1,2,3,4
2. range(2, 10) # 2 to 9
3. range(10, 0, -1) # 10 down to 1
4. enumerate(arr) # gives (index, value) pairs while looping
5. zip(arr1, arr2) # pairs elements from two lists together
6. abs(-5) # absolute value

Loops you'll use constantly

1. for i in range(len(arr)):
print(arr[i])
2. for val in arr: # when you don't need the index
print(val)
3. for i, val in enumerate(arr): # when you need both
print(i, val)
4. # Two-pointer pattern
left, right = 0, len(arr) - 1
while left < right:
left += 1
right -= 1

Recursion basics

```
def factorial(n):
```

```
    if n == 0:          # base case — ALWAYS define this first
```

```
        return 1
```

```
    return n * factorial(n - 1) # recursive case
```

Rule of thumb: always write the base case before the recursive case, or you'll get infinite recursion / stack overflow.

Linked List node (you'll define this yourself)

```
class Node:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.next = None
```

Binary Tree node

```
class TreeNode:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

Stacks & Queues

```
# Stack (LIFO) — just use a list
```

```
stack = []
```

```
stack.append(1)    # push
```

```
stack.pop()        # pop
```

```
# Queue (FIFO) — use deque, NOT list (list.pop(0) is slow)
```

```
from collections import deque
```

```
q = deque()
```

```
q.append(1)         # enqueue
```

```
q.popleft()         # dequeue
```

Things that trip up beginners

- Python has **no curly braces** — indentation IS the code block. Be consistent (4 spaces).
- `arr = arr2` doesn't copy — it points to the same list. Use `arr = arr2.copy()` or `arr = arr2[:]` to actually copy.
- Integer division: `7 // 2 = 3` (floor division). `7 / 2 = 3.5` (regular division).
- `and`, `or`, `not` — `not &&`, `||`, `!` like other languages.
- No `i++` — use `i += 1`.
- Default recursion limit is ~ 1000 — deep recursion can crash; usually not an issue at your problem sizes.

Time complexity cheat (say this out loud in interviews)

Operation	Time
• List index access <code>arr[i]</code>	• $O(1)$
• List append/pop from end	• $O(1)$
• List insert/delete at front	• $O(n)$
• Dict/Set lookup, insert	• $O(1)$ average
• Sorting	• $O(n \log n)$
• Nested loop	• $O(n^2)$

Keep this open in a tab while solving problems for the first week — you won't need to look things up by week 2 or 3, it becomes muscle memory fast.

DSA Prep Tracker (30 Days)

Mark each problem as done. Note time taken + whether you need to revisit, in the space provided.

WEEK 1: Arrays, Strings, Hashing, Basic Recursion

Arrays

- ☐ 1. Reverse an array (in-place)
- ☐ 2. Second largest element in array
- ☐ 3. Move all zeros to end
- ☐ 4. Maximum subarray sum (Kadane's algorithm) ★
- ☐ 5. Best time to buy/sell stock
- ☐ 6. Merge two sorted arrays
- ☐ 7. Find duplicate in array
- ☐ 8. Rotate array by k elements

Strings

- ☐ 9. Reverse a string / check palindrome
- ☐ 10. Check if two strings are anagrams
- ☐ 11. Longest common prefix
- ☐ 12. Count vowels/consonants, character frequency
- ☐ 13. Remove duplicate characters from string

Hashing

- ☐ 14. First non-repeating character
- ☐ 15. Two Sum (using hashmap) ★
- ☐ 16. Subarray with sum zero
- ☐ 17. Frequency of elements (word/char count)

Basic Recursion

- ☐ 18. Factorial, Fibonacci (recursive)
- ☐ 19. Sum of digits, reverse a number
- ☐ 20. Power of a number (x^n)
- ☐ 21. Check if a string is palindrome (recursively)

WEEK 2: Linked Lists, Basic Trees

Linked Lists

- [] 22. Reverse a linked list ★
- [] 23. Detect a loop (Floyd's cycle detection)
- [] 24. Find middle of linked list
- [] 25. Merge two sorted linked lists
- [] 26. Remove nth node from end

Trees (basic only)

- [] 27. Inorder/preorder/postorder traversal
- [] 28. Height of a binary tree
- [] 29. Check if two trees are identical
- [] 30. Level order traversal (BFS)

WEEK 3: Stacks, Queues, Sorting/Searching, Basic Graphs

Stacks & Queues

- [] 31. Valid parentheses (balanced brackets) ★
- [] 32. Implement stack using array/queue
- [] 33. Next greater element
- [] 34. Min stack

Sorting & Searching

- [] 35. Binary search + variants (first/last occurrence) ★
- [] 36. Bubble/selection/insertion sort (know the logic)
- [] 37. Merge sort / quicksort (understand + explain)
- [] 38. Search in rotated sorted array

Basic Graphs (light touch)

- [] 39. BFS traversal
- [] 40. DFS traversal
- [] 41. Number of islands (basic grid DFS)

WEEK 4: Revision + Mock Tests

- [] Revisit every problem marked "revisit needed" from weeks 1-3
- [] Mock test 1
- [] Mock test 2
- [] Mock test 3
- [] Mock test 4 (timed, simulate real exam conditions)
- [] Redo top 10 problems you were slowest on, untimed practice → then timed

Notes / Problems to revisit

(Use this space to jot down problem name + what tripped you up, so week 4 revision is targeted, not random)

-
-
-
-
-
-
-
-
-
-
-
-
-

Daily habit reminder

For every problem: brute force first → identify what's repeated/wasted → optimize → explain your approach out loud in one sentence before coding.

★ = shows up most frequently — prioritize these if short on time any given day.